

ARTIFICIAL INTELLIGENCE – ASSESSMENT TOOL 2

SOLUTION

QUESTION 1

*Emergency Medicine Delivery using Greedy Best-First Search and A Search**

(i) Greedy Best-First Search

Greedy Best-First Search (GBFS) selects the next node based only on the heuristic value $h(n)$, which estimates the distance to the goal. In this scenario, the drone chooses the path that appears closest to the affected area.

Strengths:

- Fast execution.
- Explores fewer nodes.
- Suitable for emergency situations.

Weaknesses:

- Ignores the actual travel cost.
 - May choose blocked or unsafe routes.
 - Does not always produce the optimal path.
-

(ii) A Search*

A* Search uses the evaluation function:

$$f(n) = g(n) + h(n)$$

where $g(n)$ is the actual travel cost and $h(n)$ is the estimated distance to the destination.

Since A* considers both travelled distance and remaining distance, it usually finds the safest and shortest route, making it more suitable for emergency medicine delivery.

(iii) Impact of Heuristic Accuracy

The heuristic function affects the performance of both algorithms.

- **Accurate heuristic:** Faster search, fewer node expansions, and better path selection.
- **Inaccurate heuristic:** More computation, incorrect route selection, and reduced efficiency.

Greedy Best-First Search depends entirely on the heuristic, whereas A* is less affected because it also considers the actual path cost.

(iv) Effect of Dynamic Changes

Roads may become blocked due to floods or bad weather.

- Greedy Search may continue following an unsuitable route.
- A* recalculates the path using updated information and selects a safer alternative.

Thus, A* performs better in dynamic environments.

(v) Recommended Algorithm

*Recommended Algorithm: A Search**

Justification:

- Finds the optimal path.
 - Considers actual cost and heuristic value.
 - Handles dynamic road conditions.
 - Provides safe and efficient navigation for emergency medicine delivery.
-

QUESTION 2

Smart Traffic Signal Optimization using Hill Climbing and Simulated Annealing / Genetic Algorithm

(i) Optimization Problem

The objective is to optimize traffic signal timings to:

- Minimize vehicle waiting time.
- Reduce fuel consumption.
- Decrease traffic congestion.

Traditional search methods are inefficient because the number of possible traffic signal combinations increases rapidly with the number of intersections.

(ii) Hill Climbing

Hill Climbing improves the current traffic signal timing by selecting a neighboring solution with a better performance.

Advantages:

- Simple to implement.
- Fast execution.
- Low memory usage.

Limitations:

- Gets trapped in local maxima.
 - Cannot escape plateaus.
 - May not find the global optimum.
-

(iii) Local Maxima, Plateaus and Ridges

- **Local Maxima:** A solution better than nearby solutions but not the best overall.
- **Plateaus:** Neighboring solutions have the same evaluation value, making it difficult to choose the next move.
- **Ridges:** The optimal solution requires multiple coordinated changes, which Hill Climbing often cannot follow.

These issues reduce the effectiveness of Hill Climbing in complex traffic optimization.

(iv) Simulated Annealing / Genetic Algorithm

Simulated Annealing occasionally accepts worse solutions to escape local maxima and continue searching for better solutions.

Genetic Algorithm maintains multiple candidate solutions and improves them using selection, crossover, and mutation. This helps discover better traffic signal timings and avoids premature convergence.

(v) Recommended Algorithm

Recommended Algorithm: Genetic Algorithm

Justification:

- Handles large search spaces.
- Optimizes multiple objectives simultaneously.
- Adapts to changing traffic conditions.
- Produces high-quality solutions suitable for smart city traffic management.

QUESTION 3

Autonomous Mars Rover using Online Search Agent

(i) Why is an Online Search Agent required?

The Mars rover operates in an unknown environment where it does not have a complete map before starting its mission. An Online Search Agent makes decisions while exploring, using sensor data to update its knowledge and choose safe paths. Unlike Offline Search, it does not require complete information in advance.

(ii) Challenges of Partial Observability and Dynamic Environment

Partial Observability:

- The rover can only sense nearby areas.
- Hidden rocks, craters, and obstacles cannot be detected until they are close.

Dynamic Environment:

- Terrain conditions may change.
 - Dust storms or new obstacles can appear unexpectedly.
 - The rover must continuously adapt its route.
-

(iii) Internal Model Update

The rover builds an internal map as it explores.

Steps:

- Sense the surroundings.
- Detect obstacles and safe paths.
- Update the internal map.
- Select the next safe move.
- Repeat until the destination is reached.

This allows the rover to navigate efficiently in unknown terrain.

(iv) Comparison with Other Search Methods

Online Search	Uninformed Search	Informed Search
Works in unknown environments	Requires complete map	Requires heuristic and map
Updates knowledge continuously	No updating	Limited updating
Suitable for dynamic environments	Poor	Moderate

Online Search is the best choice because it can make decisions while exploring.

(v) Recommended Approach

Recommended Approach: Online Search Agent

Justification:

- Handles unknown environments.
 - Updates knowledge continuously.
 - Avoids obstacles using sensor information.
 - Supports autonomous decision-making.
 - Ensures safe and efficient Mars exploration.
-

QUESTION 4

University Examination Timetable Generation using Constraint Satisfaction Problem (CSP)

(i) Variables, Domains and Constraints

Variables:

- Subjects or examinations to be scheduled.

Domains:

- Available dates.
- Time slots.
- Examination halls.

Constraints:

- No student should have overlapping exams.
 - Faculty should not have scheduling conflicts.
 - Hall capacity must be sufficient.
 - Each exam must be assigned one valid slot.
-

(ii) Backtracking Search

Backtracking assigns examination slots one by one.

If a conflict occurs, the algorithm removes the current assignment and tries another available slot until a valid timetable is generated.

Advantages:

- Finds valid schedules.
 - Systematically explores possible assignments.
 - Guarantees a solution if one exists.
-

(iii) Constraint Propagation (Forward Checking)

Forward Checking removes invalid time slots immediately after assigning an examination.

Benefits:

- Detects conflicts early.
 - Reduces unnecessary searching.
 - Minimizes backtracking.
 - Improves scheduling efficiency.
-

(iv) Real-world Challenges and Best Strategy

Challenges:

- Large number of courses.
- Limited examination halls.
- Faculty availability.
- Student examination conflicts.
- Last-minute schedule changes.

Best Strategy:

Backtracking with Forward Checking because it efficiently satisfies multiple constraints while reducing computation time.

(v) Recommended Technique

Recommended Technique: Constraint Satisfaction Problem (CSP) using Backtracking and Forward Checking.

Justification:

- Produces conflict-free timetables.
 - Handles multiple constraints efficiently.
 - Suitable for large universities.
 - Easily adapts to scheduling changes.
-

QUESTION 5

Strategic Game AI using Minimax and Alpha-Beta Pruning

(i) Minimax Algorithm

Minimax is used in two-player games where:

- **MAX Player** tries to maximize the score.
- **MIN Player** tries to minimize the score.

The AI evaluates all possible moves and selects the one with the best guaranteed outcome.

(ii) Computational Challenges

Large game trees create:

- High computation time.
- Increased memory usage.
- Slow decision making.
- Exponential growth in possible game states.

These challenges make exhaustive search impractical.

(iii) Alpha-Beta Pruning

Alpha-Beta Pruning improves Minimax by eliminating branches that cannot affect the final decision.

Advantages:

- Reduces the number of nodes evaluated.
 - Speeds up the search.
 - Produces the same optimal result as Minimax.
 - Improves real-time performance.
-

(iv) Evaluation Function

The evaluation function estimates the quality of a game state.

Factors considered:

- Number of AI units.
- Resources collected.
- Territory controlled.
- Enemy strength.
- Health of units.

A higher evaluation score indicates a better position for the AI.

(v) Recommended Strategy

Recommended Strategy: Minimax with Alpha-Beta Pruning

Justification:

- Produces optimal decisions.
- Reduces computation time.
- Explores deeper game trees.
- Suitable for real-time strategy games.
- Balances optimality and performance effectively.

